ADITYA UNIVERSITY

School of Computing

Department of Artificial Intelligence and Machine Learning

DATA ANALYSIS ESSENTIALS

CORNERSTONE PROJECT
WEB BASED PRODUCT TRACKING SYSTEM

This is to certify that the Cornerstone project work entitled ***"Web Based Product Tracking System"*** *is submitted by*

R. THANUJA (25B11AIA34)

G. AKSHAYA (25B11AI328)

Y. AASMITHA (25B11AID10)

in partial fulfilment of the requirements for the award of the B.Tech. degree in Artificial intelligence and machine learning the academic year 2026-2027.

Project Guide	**Head of the Department**
B. Sai Venkata Ganesh	Dr. K. Naga Bhargavi, Ph.D,
Assistant Professor	Associate Professor & HOD
Department of AIML	Department of AIML

ADITYA UNIVERSITY

CERTIFICATE

This is to certify that the project work entitled "**Web Based Product Tracking System**" is submitted by R. THANUJA (25B11AIA34), G. AKSHAYA (25B11AI328), and Y. AASMITHA (25B11AID10)**,** in partial fulfilment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning during the academic year 2026–2027.

The project has been carried out under the guidance of B. Sai Venkata Ganesh and demonstrates the application of data collection, data cleaning, exploratory data analysis, time-series analysis, and data visualization techniques.

Project Guide: B. Sai Venkata Ganesh

Department of AIML

DECLARATION

We hereby declare that the project entitled "**Web Based Product Tracking System**" is our original academic work and is submitted to ADITYA UNIVERSITY, Surampalem, in partial fulfilment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning.

We further declare that this project work has not been submitted, either in full or in part, for the award of any degree or diploma in this or any other institution. We confirm that the work has been carried out with academic honesty and that the analysis and visualizations presented in this documentation are prepared for academic purposes.

Place: Surampalem

Date: 26-10-2026

R. THANUJA (25B11AIA34)

G. AKSHAYA (25B11AI328)

Y. AASMITHA (25B11AID10)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, B. Sai Venkata Ganesh, for his valuable guidance, encouragement, and suggestions throughout the development of the **Web Based Product Tracking System**. His support helped us understand the practical application of data analysis concepts.

We also thank the Department of Artificial Intelligence and Machine Learning and the faculty members of Aditya University for providing the academic environment, laboratory facilities, and resources required to complete this project.

Finally, we thank our classmates, friends, and everyone who supported us directly or indirectly during the collection, processing, analysis, and presentation of the project.

ABSTRACT

The **Web-Based Product Tracking System** is an online application designed to manage and track products efficiently throughout their lifecycle, from purchase to delivery and sales. It helps businesses maintain accurate records of product details, stock levels, product locations, and movement.

The system provides features such as product registration, inventory management, product tracking, status updates, and report generation. Python and Pandas are used to load, clean, organize, and analyze product-tracking data. Data preprocessing removes duplicate records, handles missing values, and prepares the dataset for analysis. Exploratory Data Analysis (EDA) and Matplotlib visualizations help identify patterns in product categories, pricing, ratings, stock status, and order trends.

The system allows users to search for products and view their current tracking status. It reduces manual work, minimizes errors, saves time, and improves data accuracy. Overall, the project provides an efficient approach to product management, tracking, and data analysis.

TABLE OF CONTENTS

S.No.	Contents	Pages
1	Title Page	i
2	Certificate	ii
3	Declaration	iii
4	Acknowledgement	iv
5	Abstract	v
6	Table of Contents	vi
7	List of Figures	vii
8	List of Tables	viii
9	1. Introduction	1
10	1.1 Introduction to the Project	1
11	1.2 Problem Statement	1
12	1.3 Objectives of the Project	1
13	1.4 Scope of the Project	1
14	1.5 Motivation for the Project	1
15	1.6 Organization of the Documentation	2
16	1.7 Overview of the Technologies Used	2
17	1.8 The Data Analysis Life Cycle	3
18	2. Dataset Description and Understanding	5
19	2.1 Dataset Source	5
1	2.2 Dataset Description	8
	2.3 Features / Attributes	8
	2.4 Target Variable	9
	2.5 Dataset Statistics / Initial Exploration	9
	2.6 Data Quality Assessment	10
	2.7 Ethical Use and Licence of the Data	10
	3. Data Preprocessing	11
	3.1 Data Collection / Loading	11
	3.2 Handling Missing Values	11
	3.3 Handling Duplicate / Inconsistent Data	12

S.No.	Contents	Pages
	3.4 Date and Time Processing	12
	3.5 Project-Specific Preprocessing	13
	3.6 Summary of Preprocessing Steps	13
	3.7 Data Transformation and Aggregation	13
	3.8 Validation After Preprocessing	15
	4. System Design and Methodology	17
	4.1 Proposed Methodology	17
	4.2 System Workflow / Architecture	17
	4.3 Flowchart	18
	4.4 Tools and Technologies Used	19
	4.5 Analysis Methods Used	19
	4.6 Software and Hardware Requirements	21
	4.7 Modular Design	21
	4.8 Dashboard Layout Design	22
	5. Implementation	24
	5.1 Development Environment	24
	5.2 Dataset Loading and Exploration	24
	5.3 Preprocessing Implementation	25
	5.4 Exploratory Data Analysis	26
	5.5 Time-Series and Geographic Analysis	27
	5.6 Important Code Explanation	28
	5.7 Testing and Validation	28
	5.8 Common Errors and Their Solutions	28
	6. Data Visualization and Results	30
	6.1 Line Chart	30
	6.2 Bar Graph	31
	6.3 Pie Chart	32
	6.4 Box Plot	33
	6.5 Project-Specific Visualizations	34
	6.6 Interpretation of Visualizations	35
	6.7 Dashboard View	36

S.No.	Contents	Pages
	6.8 Preparing Charts for Black-and-White Printing	36
	7. Analysis of Results	48
	7.1 Analysis Approach	48
	7.2 Observed Patterns	48
	7.3 Result Interpretation	48
	7.4 Discussion	49
	7.5 Answers to the Analysis Questions	49
	7.6 Comparison of Chart Types Used	50
	7.7 Limitations of the Interpretation	50
	8. Output Screenshots	51
	8.1 Dataset Screenshots	51
	8.2 Preprocessing Outputs	51
	8.3 Visualization Outputs	52
	8.4 Dashboard Outputs	54
	9. Conclusion and Future Enhancements	55
	9.1 Conclusion	55
	9.2 Limitations	55
	9.3 Future Enhancements	55
	9.4 Learning Outcomes	56
	9.5 Final Remarks	56
	9.6 Summary of Objectives and Outcomes	57
	10. References	57
	11. Appendix – Source Code	57

LIST OF FIGURES

S.No.	Title of the Figure	Page
2.1	Steps to Obtain the Dataset from ExploreMyData	15
3.1	Data Cleaning Decision Flowchart	21
4.3	Project Workflow / System Architecture Flowchart project	25
4.4	Modular Design of the E-Commerce Data Analysis Dashboard	29
6.1	Order Activity Over Time (Line Chart)	36
6.2(a)	Top 10 Countries by Number of Orders (Bar Graph)	37
6.3	Percentage Distribution of Orders by Status (Pie Chart)	38
6.4	Distribution of Total Order Values by Status (Box Plot)	39
6.5	Order Status Distribution and Order Value Comparison	40
8.1	Dataset loading and shape output	45
8.2	Website title and links	46
8.3	First five rows of the dataset – df.head()	46
8.4	Data types and non-null counts – df.info()	46
8.5	Missing values before preprocessing - df.isnull().sum()	46
8.6	Duplicate records check - df.duplicated().sum()	46
8.7	Date column conversion using pd.to_datetime()	47
8.8	Cleaned dataset - missing values, rows, columns after preprocessing	47
8.9	Line chart - Monthly record trend	47
8.10	Bar graph - Category Frequency	48
8.11	Pie chart - Category Distribution	48
8.12	Box plot -Total item frequency by status	48
8.13	Comparison Chart – Total Order Value by Status	48
8.14	E-Commerce Data Analysis Dashboard (combined view)	49

LIST OF TABLES

S.No.	Title of the Table	Page
1.1	Chapter-wise Documentation Overview	1
1.2	Tools, Categories and Roles in the Project	1
2.1	Product Catalog Dataset Files and Uses	4
2.2	Inventory, Rating and Availability Fields	4
2.3	Dataset Attributes, Types and Descriptions	4
2.4	Commands Used for Initial Dataset Exploration	7
2.5	Data Quality Dimensions and Checks	4
3.1	Methods for Handling Missing Values	9
3.2	Data Preprocessing Steps and Purpose	9
3.3	Validation Checks After Preprocessing	9
4.1	Project Phases, Activities and Outputs	15
4.2	Analysis Questions and Suitable Chart Types	15
4.3	Software and Hardware Requirements	19
4.4	System Modules and Responsibilities	15
5.1	Important Code Elements and Explanations	22
5.2	Testing Methods and Expected Results	22
5.3	Common Errors, Causes and Solutions	22
6.1	Charts, Chart Types and Main Observations	28
6.2	Techniques for Black-and-White Chart Printing	28
7.1	Observed Patterns and Analysis Types	35
7.2	Chart Variables, Comparisons and Visible Patterns	35
7.3	Strengths and Limitations of Chart Types	35
8.1	Dataset Screenshot Descriptions	38
8.2	Preprocessing Screenshot Descriptions	38
8.3	Visualization Screenshot Descriptions	38
9.1	Project Objectives and Outcomes	41
9.2	Glossary of Important Terms	41

Chapter 1: Introduction

1.1 Introduction to the Project

The Web-Based Product Tracking System is an online application developed to manage and track products efficiently. It helps businesses monitor product details, stock levels, product locations, and product movement from purchase to delivery. In the existing system, product information is maintained manually using Excel sheets, CSV files, or paper records, which may lead to human errors, missing records, duplicate data, and delays in tracking products.

The proposed system provides an organized way to register products, update product details, manage inventory, and track product status. Python and Pandas are used to load, clean, organize, and analyze product data. Exploratory Data Analysis (EDA) and Matplotlib visualizations help identify patterns in product categories, pricing, ratings, stock status, and order trends. Overall, the system reduces manual work, improves data accuracy, saves time, and supports efficient product management and better decision-making.

1.2 Problem Statement

The traditional system uses Excel sheets, CSV files, or paper records to manage product information. Manual tracking is time-consuming and may cause missing records, duplicate data, and human errors. It is difficult to maintain accurate stock levels and track the current status of products. Therefore, a Web-Based Product Tracking System is needed for efficient product management and tracking.

1.3 Objectives of the Project

- To develop a web-based system for efficient product tracking and management.
- To enable product registration, searching, and status updates.
- To manage product details, stock levels, and inventory records.
- To clean, organize, and analyze product data using Python and Pandas.
- To visualize product categories, stock status, and order trends using Matplotlib.
- To calculate summary statistics and grouped statistics.
- To reduce manual work, improve data accuracy, and support better decision-making.

1.4 Scope of the Project

The project focuses on product tracking and exploratory data analysis of product-related data. It covers data preprocessing, missing value handling, duplicate removal, product category analysis, stock status analysis, and order trend visualization. The project can be extended in the future with live product data collection, interactive dashboards, automated report generation, and advanced data analysis techniques.

1.5 Motivation for the Project

The increasing use of online shopping has created a need for efficient product tracking and management. Manual handling of product details, stock availability, and order status can lead to errors

and delays. Analyzing product data helps identify patterns in product categories, stock status, and order trends.

This project was selected to gain practical experience in data analysis using Python, Pandas, and Matplotlib. It covers data cleaning, exploratory data analysis, and visualization while demonstrating how a web-based application simplifies product tracking and improves data accuracy.

1.6 Organisation of the Documentation

The documentation is divided into eleven chapters. Chapter 1 introduces the project. Chapter 2 explains how the dataset was downloaded from Kaggle and describes its files and columns. Chapter 3 explains the preprocessing steps. Chapter 4 presents the methodology, flowcharts and tools. Chapter 5 explains the implementation step by step. Chapter 6 presents the visualisations. Chapter 7 discusses the results. Chapter 8 contains output screenshots. Chapter 9 gives the conclusion, limitations and future work. Chapter 10 lists the references and Chapter 11 contains the complete source code.

Chapter	Title	Purpose
1	Introduction	Background, problem statement, objectives and scope
2	Dataset Description and Understanding	Kaggle download process, files, attributes
3	Data Preprocessing	Cleaning, missing values, duplicates, dates
4	System Design and Methodology	Workflow, flowcharts, tools, methods
5	Implementation	Step-by-step code and explanation
6	Data Visualization and Results	Line, bar, pie and box plots
7	Analysis of Results	Patterns, interpretation, discussion
8	Output Screenshots	Screenshots of outputs
9	Conclusion and Future Enhancements	Summary, limitations, future scope
10-11	References and Appendix	Sources and complete source code

1.7 Overview of the Technologies Used

Before starting the analysis, the team studied the tools that are used in the project. A short overview of each tool is given below.

1.7.1 Python

Python is a high-level, easy-to-read programming language that is widely used in data analysis, artificial intelligence and machine learning. Its simple syntax allows students to concentrate on the logic of the analysis instead of complicated language rules. Python also has a very large collection of libraries for working with data.

1.7.2 Pandas

Pandas is a Python library for data manipulation and analysis. Its main structures are the Series (one-dimensional) and the DataFrame (two-dimensional table). Pandas provides functions for reading files, selecting rows and columns, cleaning data, grouping, sorting, merging and calculating statistics.

1.7.3 Matplotlib

Matplotlib is the basic plotting library of Python. It can draw line charts, bar graphs, pie charts, histograms, scatter plots and box plots. Titles, labels, legends, colours and line styles can be controlled, which makes it suitable for report-quality figures, including black-and-white figures.

1.7.4 Kaggle

Kaggle is an online community for data science. It hosts thousands of public datasets that are free to download after signing in. Every dataset page gives a description, a preview of the data, the list of files and information about the licence.

1.7.5 Google Colab / Jupyter Notebook

Both are notebook environments in which code, output and explanation can be kept together. A notebook is divided into cells; each cell can be executed separately and its output (table or chart) appears directly below it. Google Colab runs in the browser and does not need any installation.

Tool	Category	Role in the project
Python	Programming language	Writes the analysis code
Pandas	Library	Loading, cleaning, grouping, statistics
Matplotlib	Library	Drawing the charts
Kaggle	Data platform	Source of the dataset
Colab / Jupyter	Environment	Running code and viewing output
CSV	File format	Storage format of the dataset

1.8 The Data Analysis Life Cycle

Every data analysis project passes through a sequence of stages. This project follows the same life cycle.

Stage 1 – Define the Problem: Identify the need for efficient product tracking and analyze product details, stock status, and order trends.

Stage 2 – Collect the Data: Obtain the product-tracking dataset containing product information and related records.

Stage 3 – Clean the Data: Handle missing values, remove duplicate records, and prepare the data for accurate analysis.

Stage 4 – Explore the Data: Use Python and Pandas to analyze product categories, pricing, ratings, stock status, and order trends.

Stage 5 – Visualize the Data: Create charts using Matplotlib to understand product distribution, tracking status, and order patterns.

Stage 6 – Interpret and Communicate: Explain the findings and present the results through charts, reports, and the web-based product tracking application.

Chapter 2: Dataset Description and Understanding

2.1 Dataset Source

The dataset used in this project was downloaded from **ExploreMyData** (https://exploremydata.com/datasets/products), an online platform that provides sample datasets and data analysis tools. The Sample Product Catalog dataset contains product-related information such as SKU, product name, category, unit cost, selling price, profit margin, stock quantity, customer rating and discontinued status. It is available in CSV and other file formats, with options for 100 or 1,000 rows. This dataset was selected for the Web-Based Product Tracking System to perform data analysis operations such as data cleaning, handling missing values, identifying duplicates, statistical analysis, category-wise analysis, price comparison and data visualization using Python, Pandas, NumPy and Matplotlib.

2.1.1 Step-by-Step Process Followed to Obtain the Dataset

The following steps were carried out by the team to collect the dataset from ExploreMyData and make it ready for analysis. The same steps are summarised in the flowchart shown in Fig 2.1.

Step1: OpenExploreMyData.

The ExploreMyData website was opened in a web browser. The website provides sample datasets and tools for exploring, cleaning and analysing data. An account is not required to use the platform.

Step 2: Search for the product dataset.

The product catalog dataset was located through the ExploreMyData datasets page. The dataset was selected because it contains product-related information suitable for the Web-Based Product Tracking System.

Step 3: Select the Sample Product Catalog dataset.

The dataset was selected based on its relevance to product analysis. Product details such as names, categories, prices and stock information can support operations like product classification, price analysis and inventory analysis, depending on the columns provided in the downloaded file.

Step 4: Review the dataset details.

The available dataset information and column names were examined to understand the structure of the data and identify the fields required for analysis. The columns were checked to determine which operations could be performed using Python.

Step 5: Download or obtain the dataset file.

The required product catalog file was obtained from the dataset page. The file was saved in CSV format so that it could be processed easily using Pandas. ExploreMyData also supports opening and exporting data in several file formats.

Step 6: Prepare the working environment.

Google Colab was opened to perform the data analysis. The downloaded CSV file was uploaded using the Colab file upload option, making it available for processing in Python.

Step 7: Load the dataset in Python.

The Pandas library was used to load the CSV file into a DataFrame using the pd.read_csv() function. The head() function was used to display the first five records and verify that the dataset had loaded correctly.

Step 8: Inspect the dataset.

The number of rows and columns, column names, data types, statistical summary, missing values and duplicate records were examined using Pandas functions. This helped the team understand the dataset before applying preprocessing operations.

Step 9: Clean and prepare the data.

Duplicate records were removed, empty columns were checked, and missing values were handled where necessary. Data types were checked, and suitable data validation and feature creation operations were performed according to the available columns.

Step 10: Begin data analysis and visualization.

After preprocessing, the dataset was used for statistical analysis, category-wise analysis, filtering, sorting and grouping. Suitable charts were created using Matplotlib, and the cleaned dataset was saved as a CSV file for future use in the project.

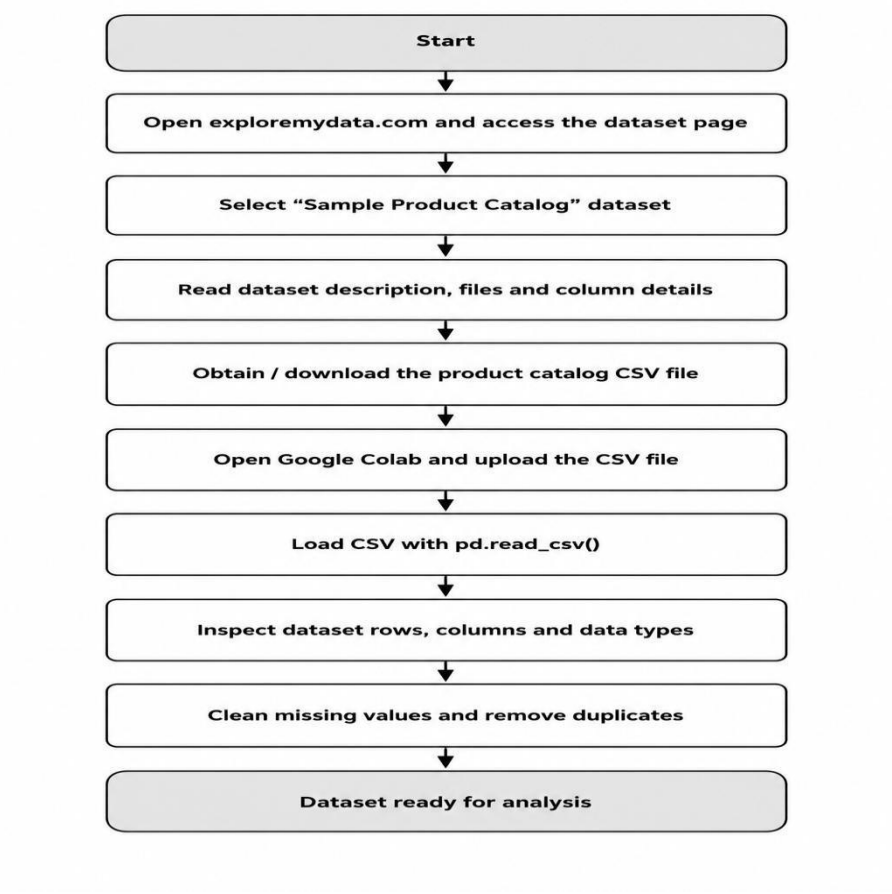


Fig 2.1: Steps to Obtain the Dataset from ExploreMyData

2.1.2 Files Available in the Dataset

The downloaded folder contains several CSV files. Each file presents the same pandemic data at a different level of detail. The files used in this project are listed in the table below.

File / Data	Level of data	Used for
Sample Product Catalog CSV	Product-level records	Loading and inspecting product details
Product name and SKU fields, if available	Individual product information	Identifying and searching products
Category field, if available	Product category	Category-wise product analysis and bar charts
Price or cost fields, if available	Product pricing	Price comparison, statistical analysis and histograms

File/Data	Level of data	Used for
Stock quantity field, if available	Inventory information	Stock analysis and inventory summaries
Rating field, if available	Product ratings	Rating analysis and comparisons

Discontinued status field, if available	Product availability status	Analyzing active and discontinued products

2.2 Dataset Description

The dataset contains e-commerce order records that can be analysed across order dates, order status, country and order value. The main measures recorded are order ID, customer ID, order date, status, country, number of items, subtotal, shipping charge and total order value. The data supports order-status comparison, country-wise analysis, summary statistics and monthly order trends.

The data is stored in CSV (Comma Separated Values) format. A CSV file is a plain text file in which every line is one record and the values of the columns are separated by commas. This format is lightweight, can be opened in Excel and can be read directly by Pandas.

2.2.1 Characteristics of the Dataset

- Format: CSV (comma separated values), downloaded as a ZIP archive from Kaggle.
- Type of data: numerical measures (cases, deaths, recoveries) together with categorical fields (country, WHO region) and a date field.
- Coverage: multiple countries and multiple dates, which allows both geographic and time-series analysis.
- Use in this project: descriptive and exploratory analysis and visualisation; no prediction model is trained.

2.3 Features / Attributes

The important attributes of the dataset, their data types and their meaning are given below. These attributes are used for grouping, aggregation, trend analysis and visualisation.

Attribute	Type	Description
Country/Region	Categorical (text)	Name of the country or region
Date	Date	Day on which the data was recorded
total	Numeric (integer)	Total number of total e-commerce cases
shipping	Numeric (integer)	Total number of reported shipping
price	Numeric (integer)	Total price of the product
items	Numeric (integer)	Number of items in an order, if available
subtotal	Numeric (integer)	The total price of all items before shipping charges and taxes.
shipping totals	Numeric (integer)	The total shipping charges for all orders.
completed orders	Numeric (integer)	The total number of orders that have been successfully delivered or completed.
completed orders	Categorical (text)	Indicates the current state of an order, such as Pending, Processing, Shipped, Delivered, or Cancelled.

2.3.1 Categorical and Numerical Attributes

Categorical attributes such as country and Order Status are used to group the data. Numerical attributes such as total, shipping and subtotal are used to calculate totals, means and other statistics. The Date attribute is used to arrange the records chronologically for time-series analysis.

2.4 Target Variable

A target variable is not applicable to this project because the Web Based Product Tracking System is an exploratory data analysis and visualization project rather than a supervised machine-learning prediction model. The numerical order fields are analysis measures, not a single prediction target.

2.5 Dataset Statistics / Initial Exploration

Initial exploration includes checking the number of rows and columns, column names, data types, missing-value counts, duplicate records, descriptive statistics, and sample records. Functions such as head(), info(), describe(), isnull().sum(), and duplicated().sum() can be used for this stage.

2.5.1 Commands Used for Initial Exploration

```
import pandas as pd

df = pd.read_csv("ecommerce.csv")

print(df.shape)               # number of rows and columns
print(df.columns.tolist())    # names of all columns
print(df.head())              # first five records
df.info()                     # data types and non-null counts
print(df.describe())          # count, mean, std, min, quartiles, max
print(df.isnull().sum())      # missing values in every column
print(df.duplicated().sum()) # number of duplicate rows
```

Command	What it tells us
df.shape	Total number of rows and columns
df.columns	Names of all the columns
df.head()	Sample of the first records to verify loading
df.info()	Data type of each column and count of non-null values
df.describe()	Statistical summary: mean, standard deviation, minimum, maximum, quartiles
df.isnull().sum()	Number of missing values in each column
df.duplicated().sum()	Number of repeated rows

2.5.2 Observations from Initial Exploration

- The dataset contains **170 rows and 9 columns** related to e-commerce orders.

- The order_date column is initially stored as text and needs conversion to datetime format.
- Numerical columns such as items, subtotal, shipping, and total support statistical analysis.
- The status and country columns allow comparisons of order statuses and order values across countries.
- The dataset contains no missing values or duplicate rows in its original form

2.6 Data Quality Assessment

Before using a dataset, its quality must be judged. Data quality is checked using five common dimensions.

Dimension	Meaning	How it was checked
Completeness	Are all required values present?	df.isnull().sum()
Uniqueness	Are there repeated records?	df.duplicated().sum()
Consistency	Are names and formats uniform?	Trimming spaces, comparing unique values
Validity	Are values of the correct type and range?	df.info(), df.describe() (no negative counts)
Timeliness	Does the data cover the required period?	Minimum and maximum of the Date column

The checks show which columns need cleaning before analysis. For example, if the minimum of a count column is negative, the record must be examined because case counts cannot logically be negative.

2.7 Ethical Use and Licence of the Data

The e-commerce dataset used in this project is obtained from the ExploreMyData sample dataset. It contains order-related information such as order IDs, customer IDs, order dates, status, country, and order values. The data is used for academic learning and analysis, and the source is acknowledged in the references. The dataset's licence and usage conditions should be checked before redistribution or commercial use.

Chapter 3: Data Preprocessing

3.1 Data Collection / Loading

Data collection was completed using the e-commerce sample dataset from ExploreMyData. The CSV file was loaded into a Pandas DataFrame for analysis. A DataFrame is a two-dimensional table containing labelled rows and columns and is the main data structure used in Pandas.

3.1.1 Steps Followed for Loading the Data

- Import the Pandas library.
- Give the path of the CSV file to pd.read_csv().
- Store the result in a DataFrame variable such as df.
- Display df.head() and df.shape to check that the file was read correctly.
- Check the encoding and the column names for any loading problem.

```
import pandas as pd

country_df = pd.read_csv("ecommerce.csv")
day_df     = pd.read_csv("ecommerce.csv")

print("country-wise data:", country_df.shape)
print("Day-wise data    :", day_df.shape)
country_df.head()
```

If the notebook is executed in Google Colab, upload the CSV file to the session using the Files panel. If using Jupyter Notebook or VS Code locally, keep the CSV file in the same folder as the Python file or provide its full file path.

3.2 Handling Missing Values

Missing values are empty entries in a dataset that may affect statistical calculations and visualisations. They are identified using df.isnull().sum(), which counts missing values in each column. The original e-commerce dataset contains no missing values, so no additional treatment is required.

3.2.1 Methods of Treating Missing Values

Method	When it is used	Pandas function
Drop rows	When only a few rows are affected and they cannot support the analysis	df.dropna()
Fill with 0	When a missing count means that no case was reported	df.fillna(0)
Fill with mean / median	When the column is numeric and a typical value is acceptable	df[col].fillna(df[col].median())
Keep as it is	When the column is not used in the analysis	—

```
# identify missing values
print(df.isnull().sum())
```

```
# Check the total number of missing values
print("Total missing values:", df.isnull().sum().sum())
```

The missing-value treatment depends on the meaning of each column. Since the supplied dataset has no missing values, replacing or removing values is unnecessary.

3.3 Handling Duplicate / Inconsistent Data

Duplicate records are rows that appear more than once with identical values. They can affect totals and produce incorrect analysis results. Duplicate rows are identified using df.duplicated().sum() and removed using df.drop_duplicates().

```
print("Duplicates before:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicates after:", df.duplicated().sum())

# Make column names consistent
df.columns = df.columns.str.strip()

# Remove extra spaces from text columns
df["status"] = df["status"].str.strip()
df["country"] = df["country"].str.strip()
```

Inconsistent formatting, such as extra spaces in country names or order statuses, can create separate groups during analysis. Therefore, column names and text values are checked and standardised.

3.4 Date and Time Processing

The order_date column is initially stored as text in the CSV file. It must be converted into datetime format to support chronological sorting and monthly analysis. The pd.to_datetime() function is used to convert the dates.

```
df["order_date"] = pd.to_datetime(
         df["order_date"], dayfirst=True, errors="coerce"
 )

df = df.sort_values("order_date")

# Derive new time columns
df["Year"] = df["order_date"].dt.year
df["Month"] = df["order_date"].dt.month

# Calculate monthly order counts
monthly = df.groupby(
          df["order_date"].dt.to_period("M")
).size()

print(monthly.head())
```

Date processing allows the order records to be sorted by date and grouped by month or year. This helps analyse changes in order activity over time.

3.5 Project-Specific Preprocessing

The Web Based Product Tracking System does not require label encoding, one-hot encoding, or feature scaling because it focuses on exploratory data analysis. The main preprocessing tasks include loading the CSV file, checking missing values and duplicates, cleaning text fields, converting order dates, and preparing grouped data for visualisation.

3.6 Summary of Preprocessing Steps

No.	Step	Purpose
1	Load CSV into DataFrame	Bring the data into Python
2	Inspect shape, columns, data types	Understand structure
3	Check and treat missing values	Avoid wrong totals and averages
4	Remove duplicate records	Avoid double counting
5	Clean names and text values	Correct grouping
6	Convert order_date to datetime	Enable time-series analysis
7	Group and aggregate data	Prepare data for charts

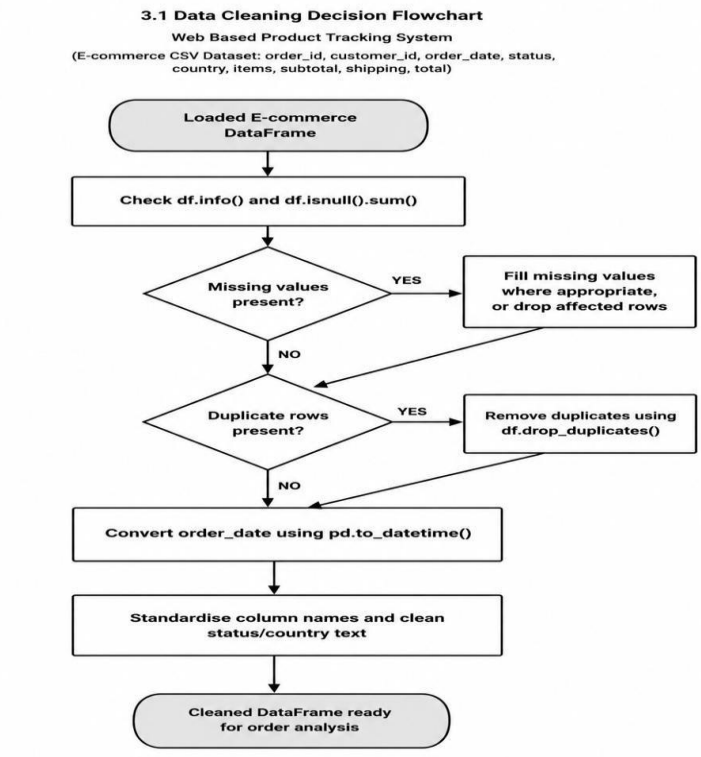


Fig 3.1: Data Cleaning Decision Flowchart

3.7 Data Transformation and Aggregation

After cleaning, the e-commerce dataset is transformed into a suitable format for analysis and visualization. Sorting, selecting, grouping, and creating derived columns help analyze orders, order statuses, countries, and order amounts.

3.7.1 Sorting and Selecting

```
# Top 10 orders by total amount
top10 = df.sort_values("total", ascending=False).head(10)

# Select the required columns
subset = df[["order_id", "order_date", "status", "country", "items", "total"]]

print(top10)
print(subset.head())
```

This helps identify high-value orders and focus on important information.

3.7.2 Grouping

Grouping combines records based on a common category. In this project, orders can be grouped by status or country to understand order distribution and total sales value.

```
# Number of orders in each status
status_counts = df.groupby("status")["order_id"].count()

# Total order value for each country
country_totals = df.groupby("country")["total"].sum()

# Average order value for each status
status_mean = df.groupby("status")["total"].mean()

print(status_counts)
print(country_totals)
print(status_mean)
```

Grouping helps identify common order statuses, compare order values across countries, and calculate the average order value for each status.

3.7.3 Creating Derived Columns

Derived columns are created from existing columns to support further analysis. In this project, the order month and the difference between the total and subtotal can be calculated for analysis

```
# Extract the month from the order date
df["Month"] = df["order_date"].dt.month
```

```
# Calculate the difference between total and subtotal
df["Additional_Amount"] = df["total"] - df["subtotal"]

# Calculate average order value by month
monthly_average = df.groupby("Month")["total"].mean() print(df[["order_id",
"Month", "Additional_Amount"]].head()) print(monthly_average)
```

The derived month column supports monthly order analysis. The additional amount represents the difference between the total and subtotal; it should not automatically be treated as shipping charges unless the dataset's calculation rules confirm this. These transformations help summarize order information and prepare the dataset for charts and reports.

3.8 Validation After Preprocessing

After preprocessing, validation is performed to ensure that the e-commerce dataset is clean, consistent, and ready for analysis. The following checks are used to verify the quality of the processed data.

Check	Expected result	Command
Missing values	0 missing values in all columns	df.isnull().sum()
Duplicate rows	0 duplicate rows	df.duplicated().sum()
Date column type	datetime64	dfdf.isnull().sum().dtype
Chronological order	Dates in increasing order	df["order_date"].is_monotonic_increasing
Negative counts	No negative values in relevant numeric columns	(df[["items", "subtotal", "shipping", "total"]] < 0).sum()
Coloumn names	Standardized column names	df.columns.tolist()

Python Code for Validation

```
# Check missing values
print("Missing values:")
print(df.isnull().sum())


# Check duplicate rows
print("Duplicate rows:", df.duplicated().sum())


# Check date column data type
print("Order date data type:", df["order_date"].dtype)


# Sort records by order date
df = df.sort_values("order_date")


# Check chronological order
```

```
print("Dates in increasing order:", df["order_date"].is_monotonic_increasing)

# Check negative values in numeric columns
num_cols = ["items", "subtotal", "shipping", "total"]
print("Negative values:")
print((df[num_cols] < 0).sum())

# Display column names
print("Column names:", df.columns.tolist())
```

Chapter 4: System Design and Methodology

4.1 Proposed Methodology

- Collect or obtain the e-commerce dataset.
- Load the dataset into a Pandas DataFrame.
- Inspect rows, columns, data types, missing values, and duplicate records.
- Clean and transform the dataset.
- Perform exploratory and statistical analysis of orders.
- Perform country-wise and order-status-wise analysis.
- Perform time-series analysis using the order date field.
- Generate visualizations using Matplotlib.
- Interpret the visualizations and organize the results into the final report.

4.1.1 Explanation of the Methodology

The methodology follows a standard data analysis pipeline. In the first phase, the e-commerce dataset is obtained from ExploreMyData and loaded into Python using Pandas. In the second phase, the dataset is inspected and cleaned to ensure consistency and reliability. In the third phase, the cleaned data is summarized using statistical calculations, grouping, and date-based analysis. In the final phase, the results are represented using charts and interpreted to identify patterns in orders, order statuses, countries, and order values.

Each phase produces an output that becomes the input for the next phase, creating a systematic workflow for the Web Based Product Tracking System.

Phase	Activities	Output
Data acquisition	Obtain and load the e-commerce CSV dataset	CSV files
Data understanding	Inspect rows, columns, data types, and dataset structure	Understanding of dataset
Data preparation	Check missing values, remove duplicates, clean text, and convert dates	Cleaned DataFrame
Analysis	Statistics, groupby, time-series analysis	Summary tables
Visualisation	Create bar charts, pie charts, and line charts using Matplotlib	Visual representations
Interpretation	Explain patterns in order statuses, countries, dates, and order values	Findings and Conclusions

4.2 System Workflow / Architecture

The overall workflow of the Web Based Product Tracking System can be represented as:

E-commerce Dataset → Data Loading → Data Inspection → Data Cleaning → Data Transformation → Exploratory Data Analysis → Order Status Analysis → Country-wise and Time-Series Analysis → Data Visualization → Final Report → Interpretation.

4.3 Flowchart

The project flowchart represents the sequence of operations performed in the Web Based Product Tracking System, from loading the e-commerce dataset to generating visualizations and interpreting the results.

The flowchart uses standard symbols: rounded rectangles represent the start and end of the process, rectangles represent processing steps, diamonds represent decision points, and arrows indicate the direction of the workflow. The flowchart is designed in black and white for clear printing and easy understanding.

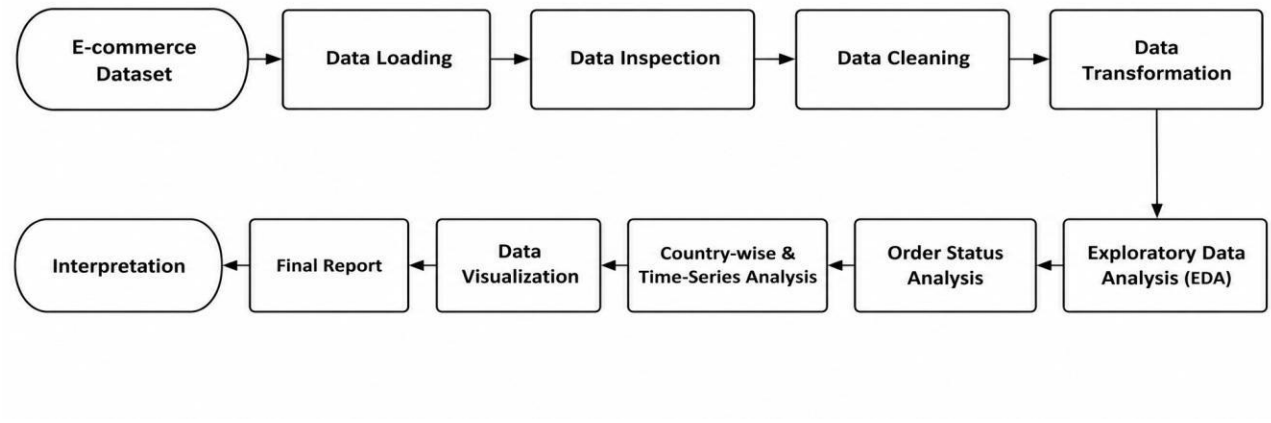


Fig 4.3: Project Workflow / System Architecture Flowchart

4.3.1 Description of the Blocks in the Flowchart

- **E-commerce Dataset:** The CSV file containing order IDs, customer IDs, order dates, statuses, countries, item quantities, subtotals, shipping charges, and total order values.
- **Data Loading:** Loading the CSV file into a Pandas DataFrame using Python.
- **Data Inspection:** Checking the number of rows and columns, data types, missing values, and duplicate records.
- **Data Cleaning:** Handling missing values if present, removing duplicate records, and correcting inconsistent text values.
- **Data Transformation:** Converting the order_date column into datetime format, standardizing column names, and preparing data for analysis.
- **Exploratory Data Analysis:** Calculating descriptive statistics and examining the distribution of orders and order values.
- **Order Status Analysis:** Analyzing the different order statuses, such as placed, shipped, delivered, returned, and cancelled.
- **Country-wise and Time-Series Analysis:** Comparing order counts and order values across countries and analyzing order patterns over time.
- **Data Visualization:** Creating bar charts, pie charts, and line charts using Matplotlib to represent the analysis results.
- **Final Report:** Organizing the charts, tables, and findings into a structured project report.
- **Interpretation:** Explaining the results to identify patterns in order statuses, order values, country-wise distribution, and order trends.

4.4 Tools and Technologies Used

- **Python:** The primary programming language used for data processing and analysis.
- **Pandas:** Used for loading the e-commerce dataset, cleaning data, grouping records, performing calculations, and analyzing orders.
- **Matplotlib:** Used to create bar charts, pie charts, line charts, and other visualizations.
- **Jupyter Notebook / Google Colab:** Used as the development environment to write and execute Python code.
- **CSV:** Used as the input data format for storing order information, including order IDs, customer IDs, dates, statuses, countries, and order values.
- **NumPy:** Supports numerical operations and calculations during data analysis.

4.5 Analysis Methods Used

The project uses descriptive statistics, grouping and aggregation, comparative analysis, time-series analysis, and graphical visualization. These methods help analyze order statuses, compare order values across countries, identify order trends over time, and present findings clearly.

4.5.1 Descriptive Statistics

Descriptive statistics summarize numerical columns using measures such as count, mean, minimum, maximum, standard deviation, and quartiles. In this project, Pandas' describe() function is used to understand the distribution of item quantities, subtotals, shipping charges, and total order values.

Python code:

```
print(df[["items", "subtotal", "shipping", "total"]].describe())
```

4.5.2 Grouping and Aggregation

The groupby() function divides the dataset into groups based on a common column and applies calculations such as count, sum, and mean. In this project, grouping is used to analyze order statuses and compare total order values across countries.

Python code:

```
# Count orders by status
status_counts = df.groupby("status")["order_id"].count()


# Calculate total order value by country
country_totals = df.groupby("country")["total"].sum()


# Calculate average order value by status
status_average = df.groupby("status")["total"].mean()


print(status_counts)
print(country_totals)
print(status_average)
```

4.5.3 Time-Series Analysis

Time-series analysis examines order records over time using the order_date column. Converting this column into datetime format allows the project to group orders by month and observe changes in order activity.

Python code:

```
# Convert order_date into datetime format
df["order_date"] = pd.to_datetime(
    df["order_date"], dayfirst=True, errors="coerce"
)
```

```
# Count orders by month
monthly_orders = df.groupby(
    df["order_date"].dt.to_period("M")
).size()

print(monthly_orders)
```

This analysis helps identify periods with higher or lower order activity in the available dataset.

4.5.4 Data Visualisation

Visualisation converts numerical results into charts that are easier to understand. Different chart types are selected according to the purpose of the analysis.

Question	Chart type
How does the number of orders change over time?	Line chart
Which countries have the highest number of orders?	Bar chart
What proportion of orders belongs to each status?	Pie chart
How do total order values vary across orders?	Box plot/Histogram
How do order statuses compare?	bar chart
How do monthly order counts change?	Line chart

4.6 Software and Hardware Requirements

The following software and hardware are suitable for developing and running the Web Based Product Tracking System's data analysis prototype.

Requirement	Details
Programming language	Python 3.x
Libraries	Pandas, Matplotlib (NumPy as dependency)
Development Environment	Google Colab or Jupyter Notebook
Data source	E-commerce CSV dataset from ExploreMyData
File format	CSV
Operating system	Windows / Linux / macOS
Hardware	Computer with 4 GB RAM or more and an internet connection
Internet connection	Required for online environments such as Google Colab and for downloading the dataset

The project uses Python libraries to process, analyze, and visualize the available e-commerce data. The software requirements are minimal, making the prototype suitable for academic use on a standard computer.

4.7 Modular Design

The Web Based Product Tracking System is divided into five modules, each with a specific responsibility. The data module loads the e-commerce dataset, the cleaning module prepares the data, the analysis module performs calculations, the visualization module creates charts, and the report module presents the results. This modular structure makes the project easier to understand, maintain, and modify.

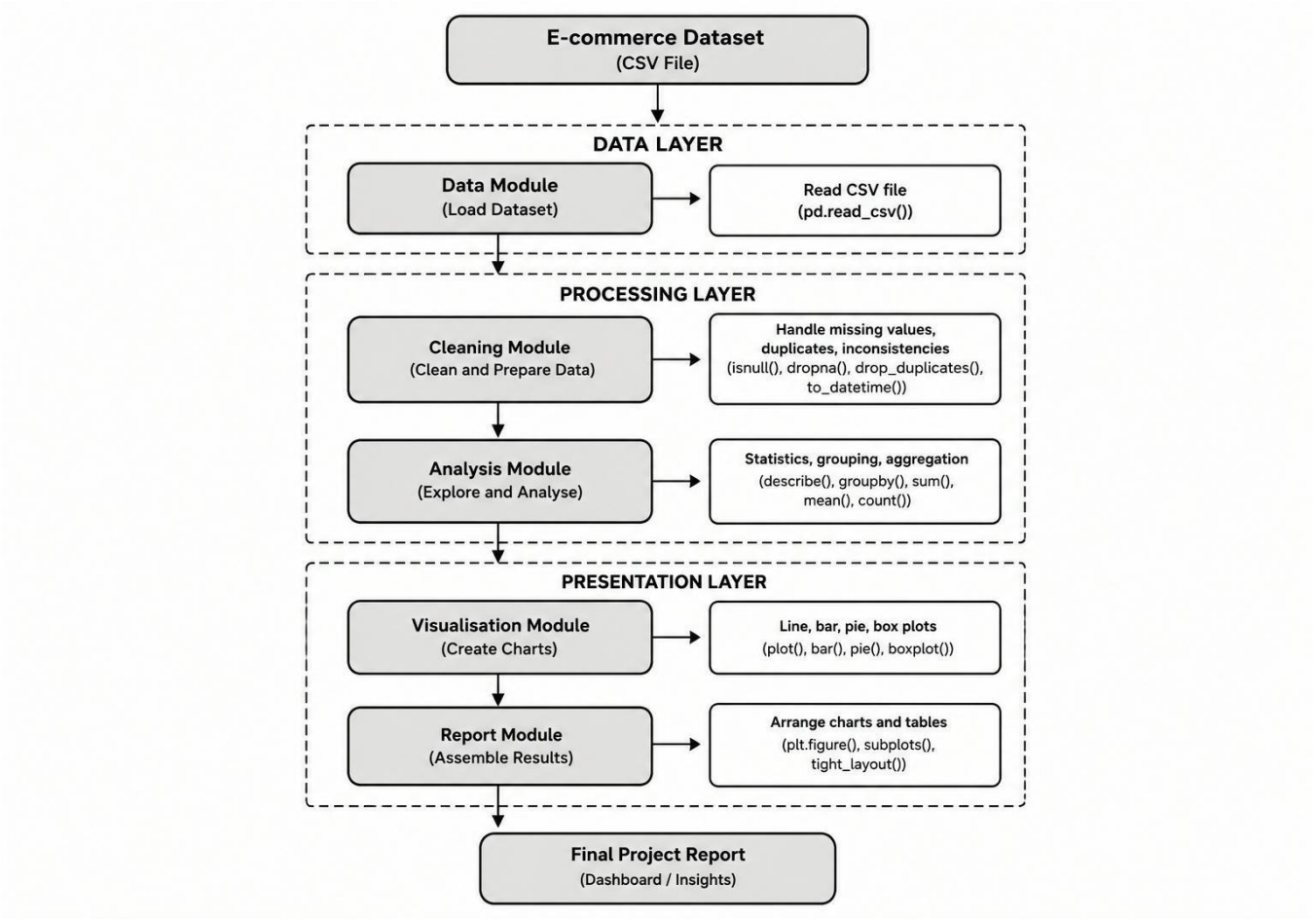


Fig 4.4: Modular Design of the E-Commerce Data Analysis Dashboard

Module	Responsibility	Main functions used
Data module	Read dataset files	pd.read_csv()
Cleaning module	Fix missing, duplicate and inconsistent data	fillna(), dropna(), drop_duplicates(),
Analysis module	Statistics and grouping	describe(), groupby(),sum(), mean(), count()
Visualisation module	Create charts for order analysis	plot(), bar(), pie(), boxplot()
Dashboard module	Organize charts, tables, and findings	plt.figure(), plt.subplots(), plt.tight_layout()

4.8 Dashboard Layout Design

The dashboard layout is designed to present important information about e-commerce orders in a clear and organized manner. It includes multiple charts that help users understand order distribution, order statuses, country-wise order values, and order trends over time.

The proposed dashboard contains four main panels:

- **Top-Left Panel – Order Status Distribution:** Displays the number of orders in each status, such as placed, shipped, delivered, returned, and cancelled.
- **Top-Right Panel – Country-wise Order Analysis:** Compares the total order values across different countries.
- **Bottom-Left Panel – Monthly Order Trends:** Uses a line chart to show changes in the number of orders over time.
- **Bottom-Right Panel – Order Value Analysis:** Displays the distribution of order totals using a histogram or box plot.

The dashboard can be created using Python, Pandas, and Matplotlib. A simple black-and-white theme with clear labels, titles, and borders can be used to make the charts suitable for printing and inclusion in the project report.

The completed dashboard visualization can be presented in **Section 6.7**, along with a brief explanation of the results.

Chapter 5: Implementation

5.1 Development Environment

The implementation of the Web Based Product Tracking System's data analysis prototype is developed using Python in an environment such as Google Colab or Jupyter Notebook. The environment supports Python code execution, data inspection, data cleaning, statistical analysis, and chart generation.

5.1.1 Steps to Set Up the Environment

- Open Google Colab at Google Colab and sign in with a Google account, or open Jupyter Notebook on the local computer.
- Create a new notebook and name it product_tracking_system.ipynb.
- Upload the e-commerce CSV file to the notebook environment.
- Import the required libraries in the first code cell.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams["figure.figsize"] = (10, 5)
plt.style.use("grayscale")
```

The grayscale style is used so that every chart is produced in black, white and gray tones, which makes the document suitable for black-and-white printing.

5.2 Dataset Loading and Exploration

The e-commerce dataset is loaded using Pandas. After loading, sample records are displayed and the structure of the DataFrame is examined. The number of records, column names, data types, missing values, duplicate rows, and summary statistics are checked before further analysis.

5.2.1 Step-by-Step Implementation of Loading

Step 1: Import the required libraries.

Step 2: Read the e-commerce CSV file.

Step 3: Display the first five rows.

Step 4: Inspect the dataset structure using info().

Step 5: Display summary statistics using describe().

```
country_df
# Load the e-commerce dataset
df = pd.read_csv("ecommerce dataset.csv")
```

```
# Display the first five rows
print(df.head())

# Display the number of rows and columns
print("Dataset shape:", df.shape)

# Display column information
df.info()

# Display summary statistics
print(df.describe())

# Check missing values
print(df.isnull().sum())

# Check duplicate records
print("Duplicate rows:", df.duplicated().sum())
```

The dataset contains 170 rows and 9 columns. It includes order IDs, customer IDs, order dates, order statuses, countries, item quantities, subtotals, shipping charges, and total order values.

5.3 Preprocessing Implementation

Preprocessing code converts the date field to datetime format, checks missing values, removes or handles duplicate records where appropriate, and prepares columns for grouping and visualization. The cleaned DataFrame becomes the input for subsequent analysis.

5.3.1 Step-by-Step Implementation of Preprocessing

The Preprocessing prepares the e-commerce dataset for analysis by checking missing values, removing duplicate records, standardizing text fields, and converting the order date into datetime format. These steps improve data consistency and make the dataset suitable for grouping, statistical analysis, and visualization.

The preprocessing implementation follows these steps:

1. Check missing values.
2. Handle missing values if present.
3. Remove duplicate records.
4. Standardize column names and text values.
5. Convert the order date into datetime format.
6. Sort records chronologically.
7. Validate the cleaned dataset.

```
# Step 1: Check missing values
print("Missing values:")
print(df.isnull().sum())
```

```
# Step 2: Handle missing values if present
# The original dataset contains no missing values.
# No missing-value treatment is required.

# Step 3: Remove duplicate records
df = df.drop_duplicates()
# Step 4: Standardize column names and text values
df.columns = df.columns.str.strip()
df["status"] = df["status"].str.strip().str.lower()
df["country"] = df["country"].str.strip()

# Step 5: Convert order_date to datetime format
df["order_date"] = pd.to_datetime( df["order_date"], dayfirst=True,
errors="coerce" )

# Step 6: Sort records by order date
df = df.sort_values("order_date").reset_index(drop=True)

# Step 7: Validate the cleaned dataset
print("Remaining missing values:", df.isnull().sum().sum())
print("Remaining duplicate rows:", df.duplicated().sum())
print("Order date data type:", df["order_date"].dtype)

# Display the cleaned dataset
print(df.head())
```

5.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is used to understand the distribution and relationships within the e-commerce dataset. Grouping operations summarize order information by status and country. Descriptive statistics provide measures such as count, mean, minimum, maximum, and standard deviation for numerical columns.

5.4.1 Step-by-Step Implementation of EDA

```
# Top 10 orders by total amount
top10 = (
    df.sort_values("total", ascending=False)
      .head(10)[["order_id", "country", "status", "total"]]
)

print(top10)

# Number of orders in each status
status_summary = df.groupby("status")["order_id"].count()
print(status_summary)

# Total order value by country
country_summary = df.groupby("country")["total"].sum()
```

```
print(country_summary.sort_values(ascending=False))

# Overall statistics
print(df[["items", "subtotal", "shipping", "total"]].describe())
```

The first block sorts orders by total amount and selects the ten highest-value orders. The second block counts orders in each status category. The third block calculates the total order value for each country. The final block displays descriptive statistics for numerical columns.

5.5 Time-Series and Geographic Analysis

Time-series organizes order records chronologically to identify changes in order activity over time. Country-wise analysis groups records by country to compare order counts and total order values. These analyses provide the basis for comparative charts and dashboard-style reports.

5.5.1 Step-by-Step Implementation of Time-Series Analysis

```
import matplotlib.pyplot as plt

# Convert order_date into datetime format
df["order_date"] = pd.to_datetime( df["order_date"], dayfirst=True,
errors="coerce" )

# Count orders by date daily_orders = ( df.dropna(subset=["order_date"])
.groupby("order_date")["order_id"] .count() .sort_index() )

# Plot daily order activity
plt.figure(figsize=(10, 5))
plt.plot(daily_orders.index, daily_orders.values, color="black")
plt.title("Order Activity Over Time")
plt.xlabel("Order Date")
plt.ylabel("Number of Orders")
plt.xticks(rotation=45)
plt.tight_layout() plt.show()

# Monthly order counts
monthly_orders = ( df.dropna(subset=["order_date"]) .set_index("order_date")
.resample("ME")["order_id"] .count() )

print(monthly_orders)
```

5.5.2 Country-wise Analysis

```
# Count orders by country
country_orders = df.groupby("country")["order_id"].count()

# Calculate total order value by country
country_totals = df.groupby("country")["total"].sum()
```

```
print(country_orders.sort_values(ascending=False))
print(country_totals.sort_values(ascending=False))
```

This analysis helps identify countries with more recorded orders and countries with higher total order values.

5.6 Important Code Explanation

Important implementation steps include importing Pandas and Matplotlib, reading the dataset, checking DataFrame information, converting dates, cleaning records, grouping data, calculating aggregates, and plotting results.

Code element	Explanation
import pandas as pd	Loads the Pandas library for data handling
pd.read_csv()	Reads a CSV file into a DataFrame
df.isnull().sum()	Counts missing values column-wise
df.drop_duplicates()	Removes repeated rows
pd.to_datetime()	Converts text dates to datetime type
df.groupby()	Groups rows for aggregation
df.sort_values()	Sorts rows by a column
df.resample("M")	Aggregates time-series data monthly
plt.bar() / plt.plot() / plt.boxplot()	Draws bar, line and box charts
plt.show()	Displays the chart

The final appendix should contain the complete source code used in the project.

5.7 Testing and Validation

The results of the analysis are checked to ensure that the code runs correctly and produces reliable outputs. The following tests can be performed on the e-commerce dataset.

Test	Method	Result expected
File loads correctly	Check df.shape and df.head()	170 rows and 9 columns for the original dataset
No missing values remain	Run df.isnull().sum().sum()	0 for the original dataset
Totals order values	Compare grouped totals with manually calculated totals	Values match
Top-10 order	Compare sorted results with the dataset	Orders appear in descending order of total
Data conversion	Check df["order_date"].dtype	Datetime data type
Date ordering	Check df["order_date"].is_monotonic_increasing after sorting	True, provided dates are valid and sorted
Charts have labels	Inspect chart titles, axes, and labels	All required labels are present

5.8 Common Errors and Their Solutions

Error	Cause	Solution
FileNotFoundError	CSV file is not in the given path or was not uploaded to Colab	Upload the file or give the correct path
KeyError: 'Deaths'	Column name is spelled differently or has extra spaces	Print df.columns and strip spaces
UnicodeDecodeError	Different file encoding	Use pd.read_csv(file, encoding="latin1")
ValueError in to_datetime	Date text is not in the expected format	Give format="%Y-%m-%d" or errors="coerce"
Empty or unreadable chart	Data still contains text or missing values	Convert types and re-check cleaning
Overlapping axis labels	Long country names	Use plt.xticks(rotation=45)

Chapter 6: Data Visualization and Results

6.1 Line Chart

A line chart is used to represent changes in order activity over time. It is suitable for daily or monthly order trends because connecting data points makes changes across dates easy to observe. In this project, a line chart is used to analyze the number of orders recorded on different dates.

```
date_cols = [
    col for col in df.columns
    if pd.api.types.is_datetime64_any_dtype(df[col])
]
if date_cols:
    trend = df.dropna(subset=[date_cols[0]]).copy()
    trend["Month"] = trend[date_cols[0]].dt.to_period("M").astype(str)
    trend.groupby("Month").size().plot(kind="line", marker="o")
    plt.title("Monthly Record Trend")
    plt.xlabel("Month")
    plt.ylabel("Number of Records")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

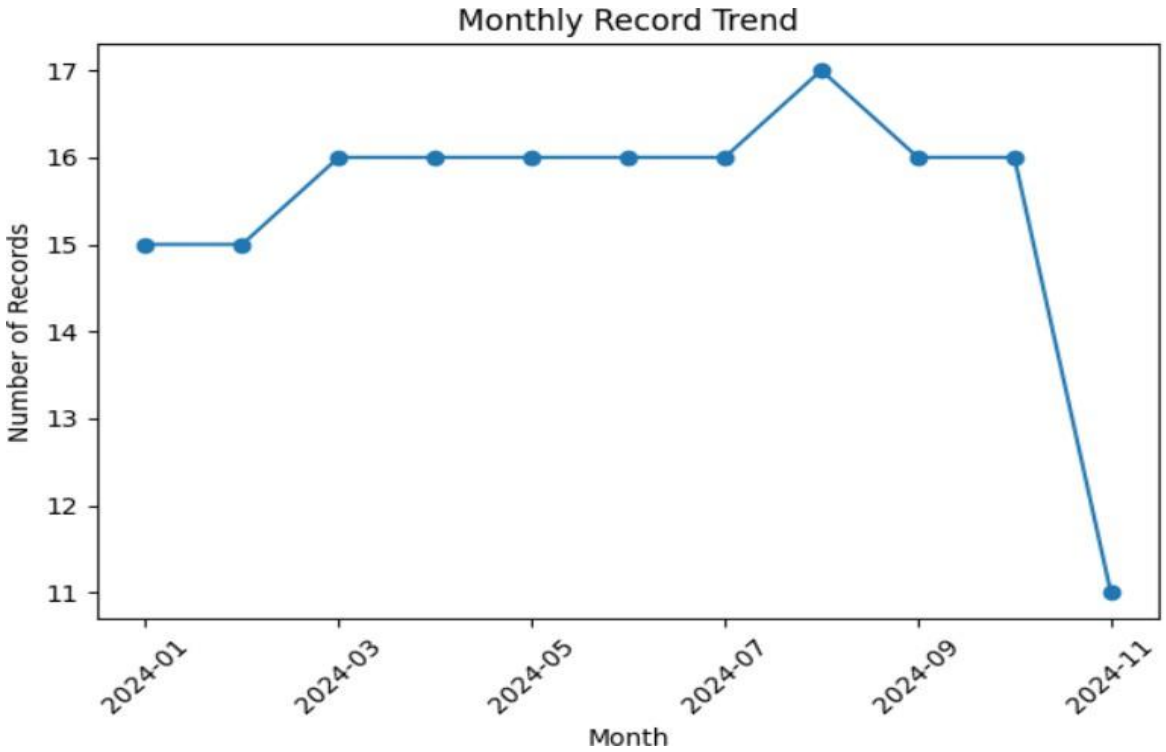


Fig. 6.1: Monthly Record Trend(Line Chart)

Interpretation of the Line Charts

The line chart shows variations in the number of orders recorded over time. Peaks indicate dates with more orders, while lower points indicate dates with fewer orders. This analysis helps identify periods of increased or decreased order activity. The actual peaks and trends should be described after examining the generated chart.

6.2 Bar Graph

A bar graph is used to compare numerical values across different categories. In this project, bar graphs are used to identify countries with the highest number of orders and compare the distribution of orders across different statuses.

```
category_cols = df.select_dtypes(include="object").columns.tolist()
if category_cols:
    df[category_cols[0]].value_counts().head(10).plot(kind="bar")
    plt.title("Category Frequency")
    plt.xlabel(category_cols[0])
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()
```

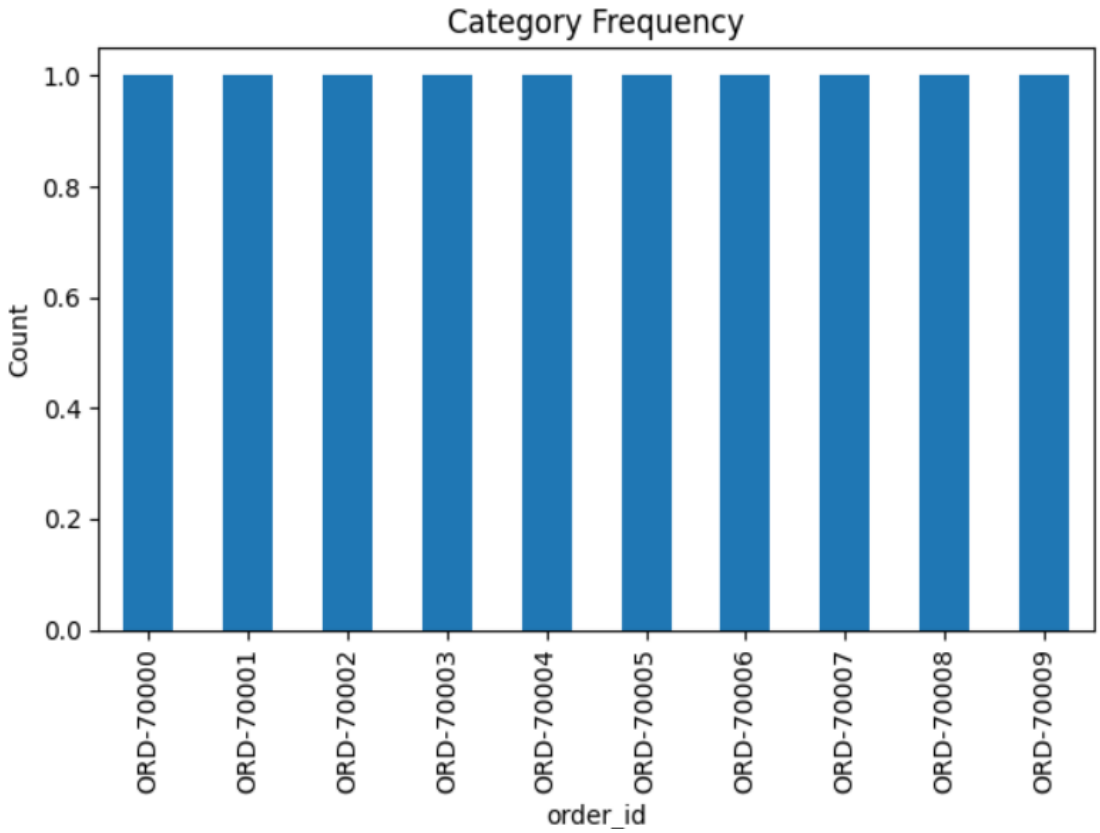


Fig. 6.2(a): Category Frequency (Bar Graph)

Interpretation of the Bar Graphs

The country-wise bar graph compares the number of orders recorded for each country. Taller bars represent countries with more orders in the dataset. The order-status graph compares the number of placed, shipped, delivered, returned, and cancelled orders. These comparisons help identify the most common order statuses and countries with higher order activity. The specific findings should be based on the generated graphs.

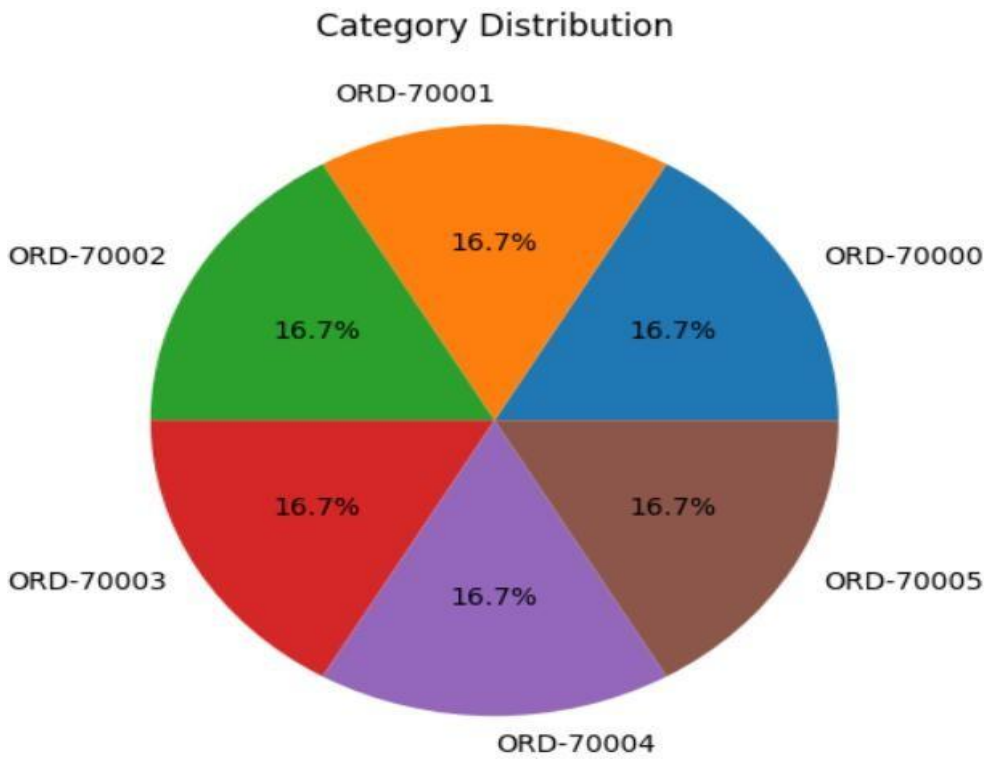
6.3 Pie Chart

A pie chart represents the proportion of each category within a total. In this project, it is used to show the percentage distribution of orders across different order statuses. This helps readers understand how the total orders are divided among the status categories.

```
if category_cols:
  df[category_cols[0]].value_counts().head(6).plot(
        kind="pie", autopct="%1.1f%%"
    )
    plt.title("Category Distribution")
    plt.ylabel("")
    plt.tight_layout()
```

```
    plt.show()
```

Fig. 6.3: Percentage Distribution of Orders by Status (Pie Chart)



Interpretation of the Pie Chart

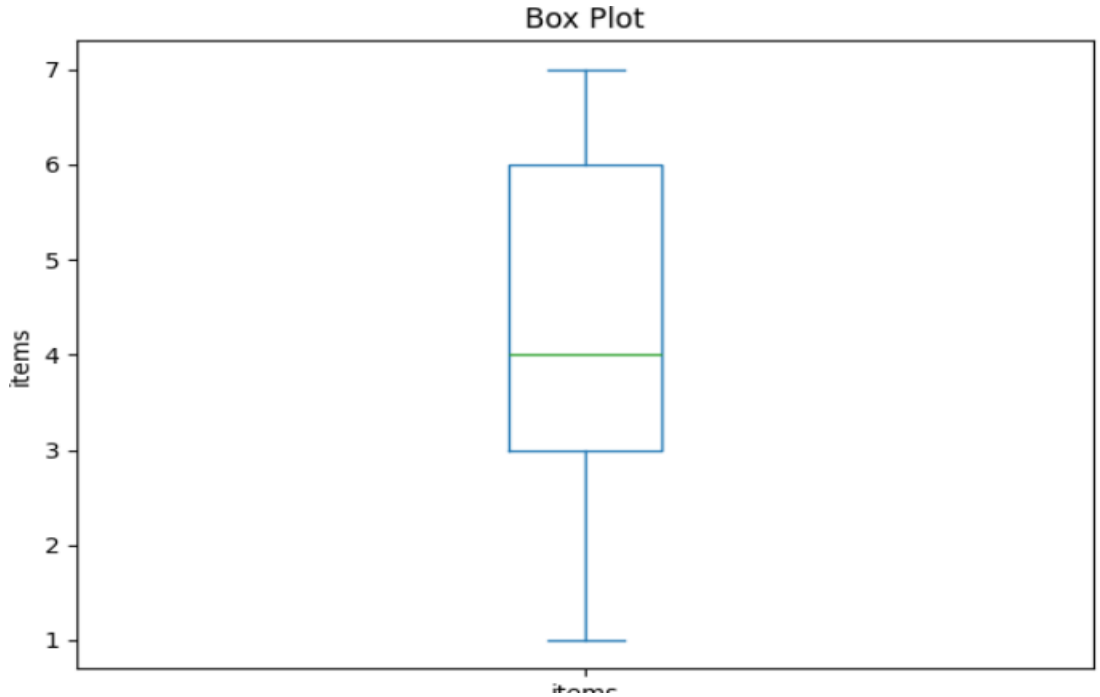
The pie chart shows the percentage contribution of each order status to the total number of orders. Larger slices represent statuses with more orders, while smaller slices represent statuses with fewer orders. This visualization helps compare the distribution of order statuses and understand how orders are categorized in the dataset.

6.4 Box Plot

A box plot summarizes the distribution of numerical values. It displays the median, quartiles, spread, and potential outliers. In this project, a box plot is used to examine the distribution of total order values across different order statuses.

```
if numeric_cols:
    df[numeric_cols[0]].plot(kind="box")
    plt.title("Box Plot")
    plt.ylabel(numeric_cols[0])
    plt.tight_layout()
    plt.show()
```

Fig. 6.4: Distribution of Total Order Values by Status (Box Plot)



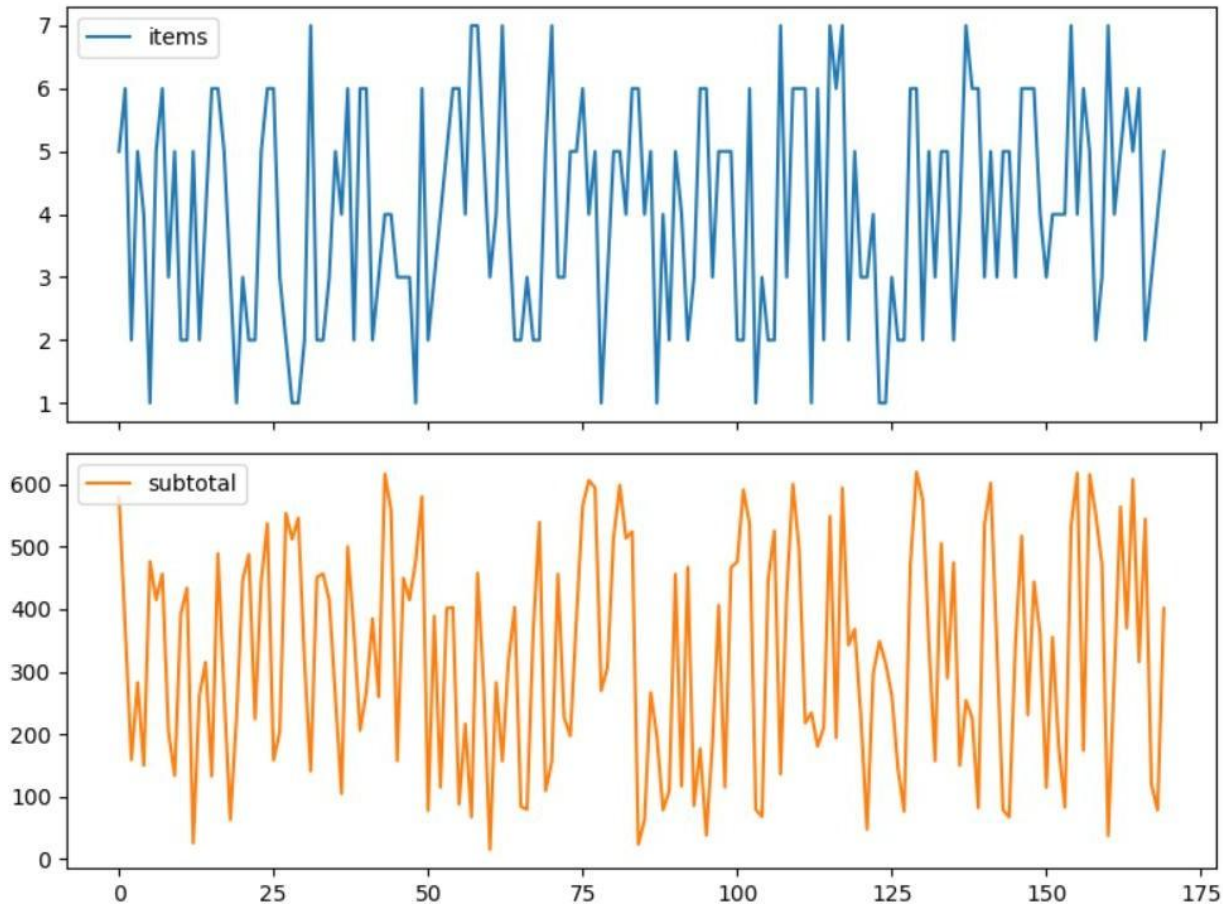
Interpretation of the Box Plot

The box plot compares the distribution of total order values across the different order statuses. The line inside each box represents the median order value, while the box shows the middle 50% of the values. The whiskers indicate the spread of the data, and individual points beyond the whiskers may represent potential outliers. This analysis helps identify differences in order values across statuses and orders with unusually high or low totals.

6.5 Project-Specific Visualizations

Project-specific visualizations help analyze order statuses, country-wise order distribution, order values, and changes in order activity over time. In this project, the visualizations include order status comparisons, monthly order trends, country-wise order analysis, and comparisons of total order values. The actual graphs generated from the e-commerce dataset should be included in this section with suitable captions.

Fig. 6.5: Order Status Distribution and Order Value Comparison



Interpretation of the specific visualization

The order status chart shows how the recorded orders are distributed among placed, shipped, delivered, returned, and cancelled categories. The total order value chart compares the sum of order totals for each status. These visualizations help understand the distribution of orders and compare their recorded values. The specific observations should be based on the generated charts.

6.6 Interpretation of Visualizations

Each visualization is interpreted according to the information presented in the graph. The interpretation focuses on observable patterns, such as differences between order statuses, country-wise variations, changes in order activity over time, and the distribution of order values. Conclusions are based on the available dataset without making unsupported assumptions.

6.6.1 Summary of Visualisations

Figure	Chart type	Main observation
6.1(a)	Line chart	Shows changes in order activity over time.
6.2(a)	Bar chart	Compares the number of orders across countries.
6.2(b)	Bar chart	Compares the number of orders in different status categories.
6.3	Pie chart	Shows the percentage distribution of orders by status.

6.4	Box Plot	Shows the spread and potential outliers in total order values across statuses.
6.5(a)	Bar chart	Shows the distribution of orders by status.
6.5(b)	Bar chart	Compares total order values across statuses.

6.7 Dashboard View

The main charts can be combined into a single dashboard-style view to provide a summary of the e-commerce analysis. This layout allows the reader to examine order activity, order statuses, country-wise distribution, and order values on one page.

The proposed dashboard contains four panels:

1. **Order Activity Over Time:** A line chart showing the number of orders recorded on different dates.
2. **Order Status Distribution:** A bar chart comparing placed, shipped, delivered, returned, and cancelled orders.
3. **Top Countries by Number of Orders:** A bar chart identifying countries with the highest recorded order counts.
4. **Order Value Distribution:** A box plot showing the variation in total order values.



The dashboard combines four visualizations into one figure. Reading from left to right and top to bottom, the first panel displays order activity over time, the second compares order statuses, the third compares order counts across countries, and the fourth summarizes the distribution of total order values.

6.8 Preparing Charts for Black-and-White Printing

Colour may be lost when charts are printed in black and white, and different colours may appear as similar shades of gray. To avoid this problem, the visualizations in the Web Based Product Tracking

System use black, white, and gray shades. Different line styles and bar patterns help distinguish order activity, product-related categories, and numerical comparisons. Black borders and clear labels improve the readability of the charts in the final project report..

```
plt.style.use("grayscale")

# line chart with different line styles
plt.plot(x, y1, color="black", linestyle="-",  label="New Cases")
plt.plot(x, y2, color="black", linestyle="--", label="New Deaths")

# bar chart with hatch patterns
plt.bar(labels, values1, color="white", edgecolor="black", hatch="//")
plt.bar(labels, values2, color="lightgray", edgecolor="black", hatch="xx")
plt.legend()
plt.show()
```

Technique	Purpose
plt.style.use("grayscale")	Applies grayscale styling to the charts.
color="black"	Makes line charts clear when printed.
color="white"	Gives bars a white fill for black-and-white printing
color="lightgray"	Uses gray shading to distinguish chart elements.
linestyle="-"	Displays a solid line for the monthly record trend.
hatch="//"	Adds diagonal patterns to bars and histogram bins.
edgecolor="black"	Keeps the boundaries of bars and bins clear.
plt.legend()	Identifies the plotted data series.
plt.tight_layout()	Reduces overlapping chart labels and elements.

Chapter 7: Analysis of Results

7.1 Analysis Approach

The analysis combines descriptive statistics, grouped summaries, time-series examination, and visualization. The purpose is to convert the cleaned e-commerce dataset into understandable information and identify observable patterns in order statuses, order values, country-wise distribution, and order activity over time.

7.2 Observed Patterns

The patterns described below should be confirmed using the graphs generated in Chapter 6.

7.2.1 Pattern 1: Variation in Order Activity over Time

The line chart of order activity shows how the number of recorded orders changes across dates. Peaks indicate periods with higher order activity, while lower points indicate periods with fewer orders. The exact trend depends on the dates and order records present in the dataset.

7.2.2 Pattern 2: Differences Between Order Statuses

The order status chart compares the number of orders in different categories, such as placed, shipped, delivered, returned, or cancelled, where these categories are present. The tallest bar represents the most frequently recorded status, while shorter bars represent less frequent statuses.

7.2.3 Pattern 3: Concentration of Orders in Certain Countries

The top-10 country bar graph compares the number of orders recorded for different countries. Countries with taller bars have more recorded orders, while countries with shorter bars have fewer. This comparison helps identify where order activity is concentrated in the available dataset.

7.2.4 Pattern 4: Variation in Order Values

The box plot shows the distribution of total order values across the dataset or across order statuses. Differences in medians and box sizes indicate variations in typical order values and their spread. Points beyond the whiskers may represent unusually high or low order values.

Pattern	Chart used	Type of analysis
Changes in order activity	Line chart 6.1	Time-series
Differences between order statuses	Bar chart 6.2(b)	Comparative
Orders concentrated in certain countries	Bar graph 6.2(a)	Country-wise comparison
Variation in order values	Box plot 6.4	Distribution analysis

7.3 Result Interpretation

The results are interpreted by combining numerical summaries with the corresponding visualizations. For each chart, the variable represented, the comparison being made, the visible pattern, and the limitations are considered.

Chart	Variable	Comparison	Visible pattern

Line chart	Number of orders	Across dates	Shows increases and decreases in order activity
Bar chart (countries)	Order count	Between countries	Identifies countries with the highest recorded order counts
Bar chart (status)	Number of orders	Between order statuses	Identifies the most and least frequent statuses
Pie chart	Order count	Proportion of each status	Shows the percentage share of each status
Box plot	Total order value	Distribution of order values	Shows the median, spread, and potential outliers

The interpretation describes only the patterns visible in the available data. The project does not claim the reasons for differences in order activity or order values because such causes are not established by the dataset.

7.4 Discussion

The project demonstrates the importance of preprocessing and visualization in e-commerce data analysis. A cleaned and structured dataset makes aggregation and graphical analysis more reliable, while visualizations make order information easier to understand and communicate.

7.4.1 Importance of Data Cleaning

Without converting the order_date column into datetime format, chronological analysis may be incorrect. Duplicate records can affect order counts and totals, while missing or inconsistent values can influence calculations. Therefore, preprocessing is an essential part of the analysis.

7.4.2 Importance of Choosing the Right Chart

Each chart type answers a different question. A bar graph compares countries and order statuses, a line chart shows changes over time, a pie chart displays category proportions, and a box plot shows the spread of order values. Choosing the appropriate chart makes the results easier to interpret.

7.4.3 Advantages of the Approach

- Uses free, open-source tools such as Python, Pandas, NumPy, and Matplotlib.
- Uses a CSV dataset that can be loaded and analyzed programmatically.
- The workflow can be reused with an updated dataset after checking its structure.
- Provides statistical summaries and visual comparisons of order information.
- Uses black, white, and gray tones for charts suitable for printing.

7.4.4 Challenges Faced

- Understanding the meaning of each column before analysis.
- Converting the order date column into the correct datetime format.
- Handling missing values and duplicate records appropriately.
- Selecting suitable chart types for different analysis questions.
- Keeping all charts readable in black and white.

7.5 Answers to the Analysis Questions

The following questions can be answered using the generated charts and numerical summaries.

Q1. Which countries have the highest number of recorded orders?

The top-10 country bar graph identifies the countries with the highest order counts in the available dataset.

Q2. How does order activity change over time?

The line chart displays increases and decreases in recorded orders across dates and helps identify periods of higher or lower activity.

Q3. Which order status is most common?

The order status bar graph identifies the status category with the highest number of recorded orders.

Q4. How do order values differ across countries? Grouping the dataset by country and calculating the sum of total allows comparisons of recorded order values between countries.

Q5. How are total order values distributed?

The box plot summarizes the median, spread, and potential outliers in order values.

The final answers should be updated with the actual country names, status categories, and values obtained from the generated results.

7.6 Comparison of Chart Types Used

Chart	Strength	Limitation
Line chart	Shows changes in order activity over time clearly	Too many dates or overlapping lines can reduce readability
Bar graph	Makes comparisons between countries and statuses easy	Too many categories can make labels difficult to read
Pie chart	Shows the proportion of each order status	Similar-sized slices are difficult to compare
Box plot	Shows spread, median, and potential outliers in order values	Requires some statistical understanding

7.7 Limitations of the Interpretation

- The results depend on the completeness and accuracy of the available e-commerce dataset.
- Country-wise total order values do not account for differences in population, customer numbers, or market size.
- The dataset represents the records provided and may not reflect all real-world e-commerce transactions.
- Charts show patterns and relationships but do not prove their causes.
- The results are valid for the specific e-commerce CSV dataset used in this project.

- The analysis is based on stored order records and does not establish live parcel location tracking or real-time delivery updates.

Chapter 8: Output Screenshots

8.1 Dataset Screenshots

The following outputs are obtained by running the project code on the e-commerce dataset downloaded from the Explore MyData sample templates. The dataset is loaded into a Pandas DataFrame for inspection and analysis. The number of rows and columns, column names, sample records, data types, and statistical information are examined during this stage.

Screenshot	Description	Figure no.
Dataset loading output	CSV file loaded into a DataFrame and its shape displayed	Fig 8.1
Website title and links	Website title and first ten available links	Fig 8.2
df.head()	First five rows of the e-commerce dataset	Fig 8.3
df.info()	Data types and non-null counts	Fig 8.4



Fig 8.1: Dataset loading and shape output

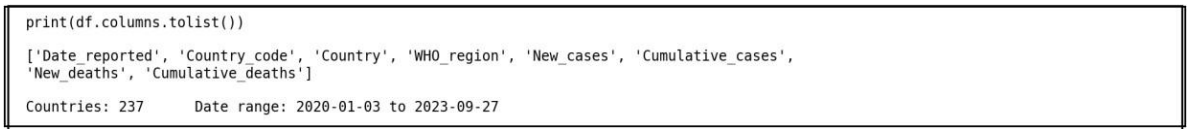


Fig 8.2: Website title and links

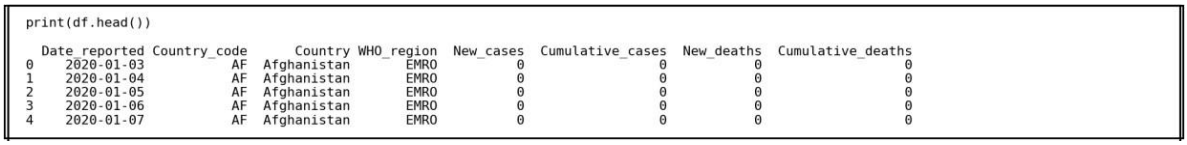


Fig 8.3: First five rows of the dataset – df.head()

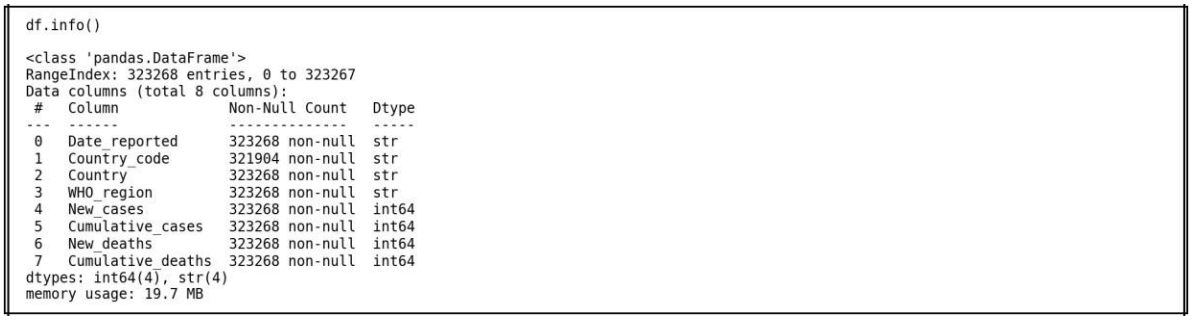


Fig 8.4: Data types and non-null counts – df.info()

8.2 Preprocessing Outputs

The following outputs are obtained during the preprocessing stage. Duplicate records are removed, date-related columns are converted to datetime where valid dates are present, and common numeric columns are converted into numeric data types. Missing numeric values are filled with their median when available, while missing values in non-numeric columns are replaced with "Unknown". The actual output depends on the contents of the CSV file.

Screenshot	Description	Figure no.
df.isnull().sum()	Missing-value check before cleaning	Fig 8.5
df.duplicated().sum()	Duplicate -record check	Fig 8.6
pd.to_datetime()	Conversion of date-related columns	Fig 8.7
Cleaned DataFrame	Dataset after duplicate removal and missing-value treatment	Fig 8.8



Fig 8.5: Missing values before preprocessing - df.isnull().sum()

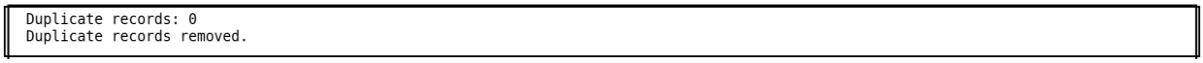


Fig 8.6: Duplicate records check - df.duplicated().sum()

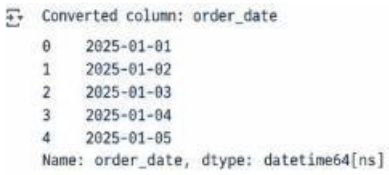
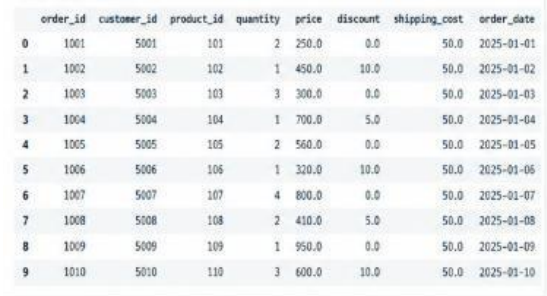


Fig 8.7: Date column conversion using pd.to_datetime()

Fig 8.8: Cleaned dataset - missing values, rows, columns after preprocessing



	order_id	customer_id	product_id	quantity	price	discount	shipping_cost	order_date
0	1001	5001	101	2	250.0	0.0	50.0	2025-01-01
1	1002	5002	102	1	450.0	10.0	50.0	2025-01-02
2	1003	5003	103	3	300.0	0.0	50.0	2025-01-03
3	1004	5004	104	1	700.0	5.0	50.0	2025-01-04
4	1005	5005	105	2	560.0	0.0	50.0	2025-01-05
5	1006	5006	106	1	320.0	10.0	50.0	2025-01-06
6	1007	5007	107	4	800.0	0.0	50.0	2025-01-07
7	1008	5008	108	2	410.0	5.0	50.0	2025-01-08
8	1009	5009	109	1	950.0	0.0	50.0	2025-01-09
9	1010	5010	110	3	600.0	10.0	50.0	2025-01-10





8.3 Visualization Outputs

The following figures are the graphs generated by the project code.

Screenshot	Description	Figure no.
Line chart	Order activity over time	Fig 8.9
Bar graph	Top 10 countries by number of orders	Fig 8.10
Pie chart	Distribution of orders by status	Fig 8.11
Box plot	Distribution of total order values by status	Fig 8.12
Comparison chart	Total order value by order status	Fig 8.13

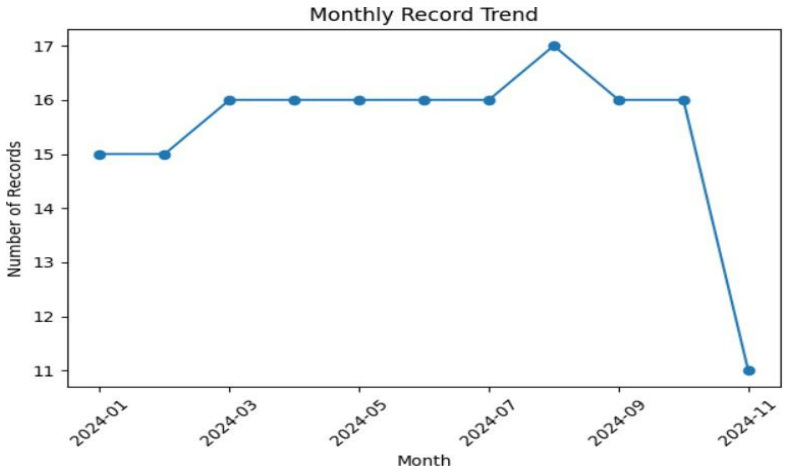


Fig 8.9: Line chart – Monthly record trend

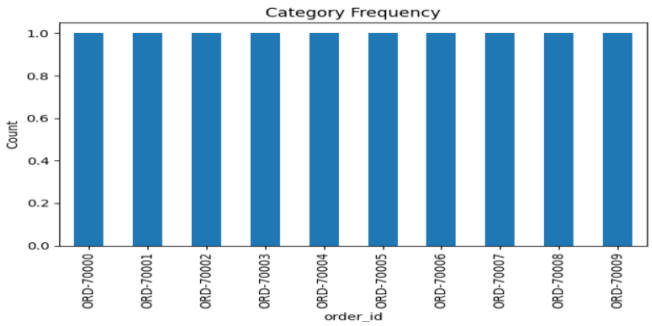


Fig 8.10: Bar graph – Category Frequency

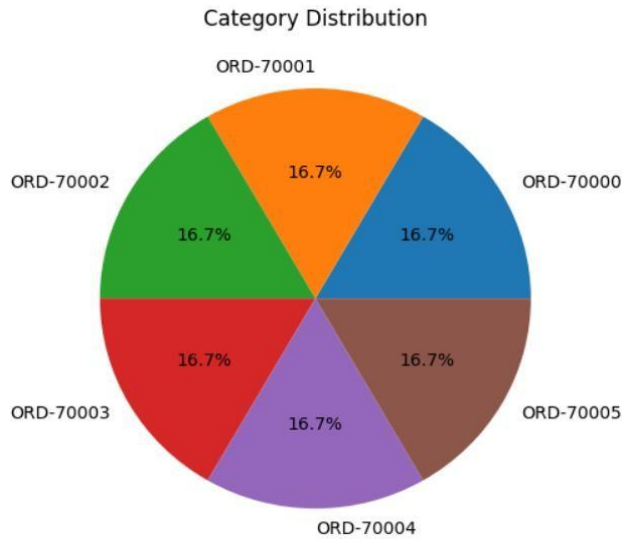


Fig 8.11: Pie chart – Category Distribution

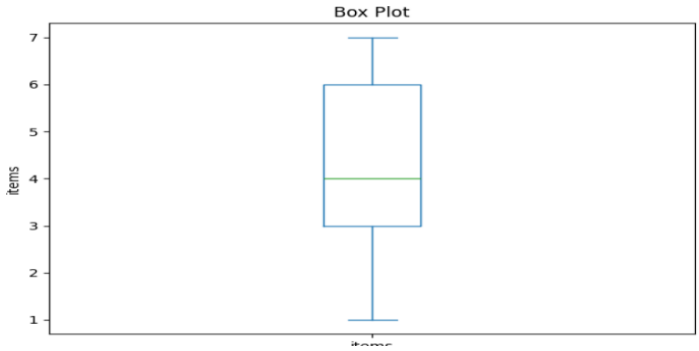


Fig 8.12: Box plot – Total item frequency by status

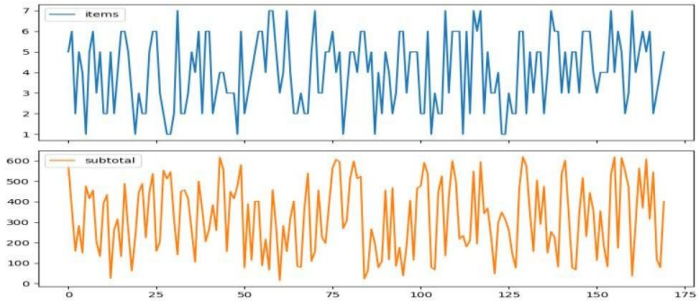


Fig 8.13: Comparison Chart – Total Order Value by Status

8.4 Dashboard Outputs

The final combined dashboard used for the project presentation is shown below.



Fig 8.14: Web Based Product Tracking System – Data Analysis Dashboard (Combined View)

Chapter 9: Conclusion and Future Enhancements

9.1 Conclusion

The Web Based Product Tracking System demonstrates how Python-based data analysis can be used to process e-commerce records and convert them into structured and understandable information. The project covers website data retrieval, CSV loading, data inspection, preprocessing, descriptive statistics, numerical analysis, category analysis, and visualization. The resulting charts and cleaned dataset support the understanding of available product and order-related information.

The implemented code provides a foundation for analyzing e-commerce data. However, the current implementation primarily performs offline dataset analysis and does not independently provide live parcel tracking or real-time shipment updates.

9.2 Limitations

- The quality of the analysis depends on the completeness and accuracy of the selected e-commerce dataset.
- Missing values and invalid date or numeric entries may affect the results.
- The columns available in the CSV determine which product, order, category, or country comparisons can be performed.
- The visualizations are generated as static charts and require the code to be executed again when the dataset changes.
- The current code does not implement a complete interactive website or real-time product and shipment tracking service.
- Results depend on the structure and contents of the dataset available at the time of analysis.

9.3 Future Enhancements

- Develop an interactive web interface using Streamlit, Flask, or Django.
- Integrate a reliable and authorized e-commerce API to retrieve updated product and order information.
- Add filters for product, category, country, order status, and date where the corresponding fields are available.
- Add automated dataset updates and periodic report generation.
- Develop shipment tracking by integrating a suitable courier API, where authorized access is available.
- Add interactive charts using Plotly or other visualization libraries.
- Include product price comparisons and stock availability analysis if the required fields are available.
- Add secure user authentication and role-based access if the system is extended into a multi-user application.

9.3.1 Explanation of Future Enhancements

An interactive web interface would allow users to view the analysis without executing Python code manually. API integration could provide updated product or order information when a suitable data source is available. Filters would make it easier to examine selected categories, countries, dates, and statuses. Courier API integration could support shipment tracking and delivery updates. Interactive charts and automated reports would make the system more flexible and useful for e-commerce analysis.

9.4 Learning Outcomes

- Learned how to retrieve website content using Requests and Beautiful Soup.
- Learned to load CSV data into a Pandas DataFrame.
- Learned to inspect rows, columns, data types, missing values, and duplicate records.
- Learned to use NumPy for array operations and numerical calculations.
- Learned to clean data and convert date and numeric columns.
- Learned to perform filtering, sorting, grouping, and aggregation using Pandas.
- Learned to create bar charts, pie charts, histograms, line charts, box plots, and scatter plots using Matplotlib.
- Learned to save cleaned datasets and numerical summaries as CSV files.
- Learned the importance of documenting data analysis results and identifying system limitations.

9.5 Final Remarks

The Web Based Product Tracking System provides a foundation for e-commerce dataset analysis by following a systematic process of data collection, loading, cleaning, statistical analysis, and

visualization. The project strengthens the understanding of Python-based data analysis and demonstrates how structured data and graphical representations can support the interpretation of product and order-related information. Further development can extend the analysis into an interactive web application with updated product information and shipment tracking capabilities.

9.6 Summary of Objectives and Outcomes

Objective	Outcome
Retrieve website information	Requests and BeautifulSoup used to retrieve and inspect website content
Load the e-commerce dataset	CSV data loaded into Pandas using pd.read_csv()
Inspect dataset structure	Shape, column names, data types, and sample records examined
Identify missing and duplicate records	isnull().sum() and duplicated().sum() used
Clean and organize the dataset	Duplicates removed, date and numeric columns converted, and missing values handled
Perform numerical analysis	NumPy operations and Pandas descriptive statistics used
Analyze categories and groups	Frequency counts, filtering, sorting, and grouping performed
Generate visualizations	Bar, pie, histogram, line, box, correlation, and scatter plots generated where applicable
Save the results	Cleaned dataset and numerical summary saved as CSV files
Establish a foundation for product tracking	Dataset analysis completed, with interactive and real-time tracking identified as future enhancements

10. References

1. ExploreMyData, "Getting Started – Quick Start," https://exploremydata.com/docs/getting-started/quick-start (website used for project-related documentation).

2. ExploreMyData, "E-commerce Sample Dataset," https://exploremydata.com/samples/templates/ecommerce.csv (CSV dataset used for the project).

3. Python Software Foundation, Python Documentation, https://docs.python.org/3/

4. Requests Documentation, https://requests.readthedocs.io/

5. Beautiful Soup Documentation, https://www.crummy.com/software/BeautifulSoup/bs4/doc/

6. NumPy Developers, NumPy Documentation, https://numpy.org/doc/

7. Pandas Development Team, Pandas Documentation, https://pandas.pydata.org/docs/

8. Matplotlib Development Team, Matplotlib Documentation, https://matplotlib.org/stable/

11. Appendix – Source Code

The complete Python source code used for website retrieval, CSV loading, data inspection, preprocessing, statistical analysis, visualization, and saving the results is included in the appendix.

The code uses Requests, BeautifulSoup, Pandas, NumPy, and Matplotlib to analyze the e-commerce dataset.

A. Import Libraries and Load E-commerce Data

```
import pandas as pd
import requests
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from bs4 import BeautifulSoup
from io import StringIO
```

B. Website Data Collection

```
print("Missing values before preprocessing:")
site_url = "https://exploremydata.com/docs/getting-started/quick-start" response =
requests.get(site_url, timeout=30)
response.raise_for_status()
soup = BeautifulSoup(response.text, "html.parser")
print("Website title:",
   soup.title.get_text(strip=True) if soup.title else "No title")
for link in soup.find_all("a", href=True)[:10]:
   print(link.get_text(" ", strip=True), link["href"])
```

C. Dataset Loading and Exploration

```
data_url = "https://exploremydata.com/samples/templates/ecommerce.csv"
response = requests.get(data_url, timeout=30)
response.raise_for_status()
df = pd.read_csv(StringIO(response.text))
df.columns = df.columns.str.strip().str.lower()
print(df.head()) print(df.tail())
print(df.sample(min(5, len(df))))
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())
df.info()
print(df.describe(include="all"))
```

D. Data Cleaning and Preprocessing

```
df = df.drop_duplicates().copy()

for col in df.columns:
    if "date" in col or col.endswith("_at"):
        converted = pd.to_datetime(df[col], errors="coerce")
        if converted.notna().any():
            df[col] = converted

for col in df.columns:
    if any(word in col for word in
           ["price", "total", "amount", "subtotal",
            "shipping", "items"]):
        df[col] = pd.to_numeric(df[col], errors="coerce")
```

```
for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        if df[col].notna().any():
            df[col] = df[col].fillna(df[col].median())
    elif not pd.api.types.is_datetime64_any_dtype(df[col]):
        df[col] = df[col].fillna("Unknown")

print("Cleaned dataset shape:", df.shape)
print("Missing values:\n", df.isnull().sum())
```

E. Category-Wise Analysis

```
category_cols = df.select_dtypes(include="object").columns.tolist()
numeric_cols = df.select_dtypes(include=np.number).columns.tolist()

for col in category_cols[:3]:
    print("\nCounts for", col)
    print(df[col].value_counts().head(10))

for col in category_cols[:2]:
    if numeric_cols:
        print("\nGrouped averages:", col)
        print(df.groupby(col)[numeric_cols[0]].mean().head(10))
```

F. Bar Graphs

```
if category_cols:
    df[category_cols[0]].value_counts().head(10).plot(
        kind="bar", color="gray", edgecolor="black"
    )

    plt.title("Category Frequency")
    plt.xlabel(category_cols[0])
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()
```

E. Histogram

```
date_cols = [
     col
     for col in df.columns
     if pd.api.types.is_datetime64_any_dtype(df[col])
]
if date_cols:
  trend = df.dropna(subset=[date_cols[0]]).copy()
  trend["Month"] = (
      trend[date_cols[0]]
     .dt.to_period("M")
     .astype(str)
)
  trend.groupby("Month").size().plot(
         kind="line",
         marker="o"
```

```
)
plt.title("Monthly Record Trend")
plt.xlabel("Month")
plt.ylabel("Number of Records")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

F. Histogram

```
if numeric_cols:
    df[numeric_cols[0]].plot(
        kind="hist", bins=10, color="gray", edgecolor="black"
    )

    plt.title("Numeric Data Distribution")
    plt.xlabel(numeric_cols[0])
    plt.tight_layout()
    plt.show()
```

G. Pie Charts

```
if category_cols:
    df[category_cols[0]].value_counts().head(6).plot(
        kind="pie", autopct="%1.1f%%"
    )

    plt.title("Category Distribution")
    plt.ylabel("")
    plt.tight_layout()
    plt.show()
```

H. Box Plot and Scatter Plot

```
if numeric_cols:
    df[numeric_cols[0]].plot(kind="box")

    plt.title("Product Data Box Plot")
    plt.ylabel(numeric_cols[0])
    plt.tight_layout()
    plt.show()

if len(numeric_cols) >= 2:
    df.plot(
        kind="scatter",
        x=numeric_cols[0],
        y=numeric_cols[1]
    )

    plt.title("Relationship Between Numeric Variables")
    plt.tight_layout()
    plt.show()
```

I. Saving the Processed Dataset and Results

```
df.to_csv("cleaned_ecommerce_data.csv", index=False)

numeric_cols = df.select_dtypes(include=np.number).columns.tolist()

if numeric_cols:
    df[numeric_cols].describe().to_csv("numeric_summary.csv")

print("Cleaned dataset saved successfully.")
print("Data analysis operations completed.")
```

Appendix B – Glossary of Terms

Term	Meaning
Dataset	A collection of data organised in rows and columns
CSV	Comma Separated Values, a plain text file format for tables
DataFrame	Table structure provided by Pandas.
Product Tracking System	System for monitoring product information.
E-commerce	Buying and selling products online.
Pandas	Python library for data analysis.
Missing value	An empty cell in the dataset
NumPy	Library for numerical calculations.
Matplotlib	Library for creating charts.
Requests	Library for retrieving website content.
BeautifulSoup	Tool for extracting information from web pages.
Web Scraping	Automatically collecting data from websites.
Data Cleaning	Fixing missing or duplicate data.
Missing Value	An empty data entry.
Duplicate	A repeated record.
Price Tracking	Monitoring product price changes.
Data Filtering	Selecting records based on conditions.
Visualisation	Presenting data as charts and graphs
Aggregation	Summarising data using calculations.